

Hola , me.siwnto un pcoo perdido y frustrado. Estudie ing de ia pero no vi rag y todo lo que preguntan es rag. El tema es que es bastante extenso y se usa en varias cloud. Lo que mas me preguntan es de aws y luego de gcp. Y luego hay metodologías con Lang chain y directo en bedrock que e es el.que mas se usa. El tema es que siento que no fluyó cuando le pido a una ia que me de un rproyecto comoeto que cubra todo. (Quiza igual no cubre todo el ciclo de vida preofional(y a la.funl me siento como periddo del camino. Segir. Me.gustaria tener el cami o profesional del tema, ya sea en video, libros,

Show more ▾

Worked for 3m 11s >

Dani, primero: **no hiciste el ridículo**. Que Bedrock tenga una pestaña llamada *Guardrails* no significa que un ingeniero profesional simplemente deba saber dónde hacer clic. En una entrevista quieren saber si comprendes el problema: controlar entradas y salidas, detectar prompt injection, proteger información sensible, limitar temas, reducir alucinaciones, controlar las herramientas del agente y registrar lo ocurrido.

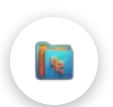
Actualmente Bedrock Guardrails permite configurar filtros de contenido, ataques al prompt, temas prohibidos, palabras, datos sensibles, verificación de contextual grounding y automated reasoning. Pero eso es solamente la implementación administrada de AWS; la arquitectura de seguridad es más amplia.  AWS Documentat... +3

Tú no estás empezando desde cero. Ya tienes Python, machine learning, Docker, FastAPI, GCP y experiencia en proyectos empresariales. Lo que te falta no es "aprender IA"; te falta **un mapa ordenado de AI Engineering aplicado a RAG**.

Por qué te estás sintiendo perdido

Estabas intentando aprender simultáneamente tres cosas diferentes:

1. **RAG como arquitectura:** embeddings, chunking, recuperación, vector databases, reranking, generación, evaluación.



2. **LangChain como framework:** loaders, retrievers, chains, LangGraph y observabilidad con LangSmith.
3. **Bedrock como plataforma:** modelos, Knowledge Bases, Guardrails, Agents, Flows, evaluaciones, Prompt Management, IAM, logging, despliegue y AgentCore.

Un proyecto generado por una IA puede enseñarte a conectar componentes, pero normalmente no te explica sistemáticamente las decisiones, alternativas, errores, seguridad, evaluación y operación. Bedrock ya tiene una superficie bastante extensa: Knowledge Bases, Guardrails, evaluaciones, Prompt Management, Flows, Agents y AgentCore son capacidades separadas que forman parte del ciclo profesional.

 AWS Documentat... +4

El orden correcto para ti es:

RAG sin framework → Bedrock directamente → producción y seguridad → LangChain → GCP por equivalencia.

No AWS, GCP y LangChain al mismo tiempo.

La combinación de cursos que escogería para ti

No buscaría un único curso "que lo tenga todo". Usaría tres recursos con funciones diferentes.

1. Curso principal de Amazon Bedrock

Mi elección actual sería **[2026] Complete AWS Bedrock Generative AI Course + Projects**, de Patrik Szepesi.

Está actualizado a junio de 2026, tiene aproximadamente 15 horas y 41 minutos, más de 16.000 estudiantes y actualmente aparece con 4,5/5. Incluye Guardrails, evaluación, Prompt Management y versionamiento, cuotas, inference profiles, serverless deployment, AgentCore, memoria, agentes, observabilidad y proyectos. Es de los que mejor cubre la amplitud actual de Bedrock.  Udemy

No lo consideraría suficiente para dominar RAG profundamente, pero sí como tu **mapa general de Bedrock**.

2. Curso principal de RAG

Usaría **Ultimate RAG Bootcamp Using LangChain, LangGraph & LangSmith**, de Krish Naik.

Está actualizado a mayo de 2026, tiene más de 25.000 estudiantes y una valoración actual de 4,6/5. Cubre RAG tradicional, recuperación avanzada, búsqueda híbrida, multimodal RAG, Agentic RAG, LangGraph y observabilidad con LangSmith.  Udemy

Este curso te da profundidad en RAG, pero debes evitar convertirte en alguien que solo sabe escribir código de LangChain. Cada vez que aparezca una abstracción, debes preguntarte:

¿Qué está haciendo realmente esta clase por debajo?

3. Ruta oficial de AWS

Completaría el **Generative AI Learning Plan for Developers**, de AWS Skill Builder. La versión actual incluye laboratorios, dura aproximadamente 20 horas y fue actualizada en abril de 2026. Sirve para contrastar lo aprendido en Udemy con la forma oficial en que AWS espera que entiendas sus servicios.  AWS Skill Builder

No necesitas consumir los tres recursos linealmente. El curso de Bedrock será tu columna vertebral; el curso de RAG profundizará cada concepto; Skill Builder y la documentación oficial serán tu validación.

Como libro de referencia, no para leer obligatoriamente de principio a fin, escogería **AI Engineering**, de Chip Huyen. En particular, los capítulos de metodología de evaluación, evaluación de sistemas y RAG/agentes. El capítulo de RAG incluye arquitectura, algoritmos de recuperación, optimización, agentes, herramientas, memoria, fallos y evaluación.  O'Reilly Media +2

Tu camino profesional de 12 semanas

Semanas 1 y 2: comprender RAG sin LangChain

Construye un RAG pequeño utilizando Python directamente:

Documentos → extracción → chunks → embeddings → vector store → top-k retrieval → prompt → modelo → respuesta con citas.

Debes experimentar con tamaño de chunk, overlap, top-k y distintas consultas. En esta etapa necesitas poder explicar:

- Por qué un chunk muy pequeño pierde contexto.
- Por qué uno demasiado grande introduce ruido y aumenta tokens.
- Qué diferencia hay entre embeddings y el modelo generativo.

- Qué guarda realmente una vector database.
- Por qué recuperar cinco fragmentos no significa que los cinco sean relevantes.
- Cuándo usar búsqueda semántica, keyword o híbrida.

El resultado debe ser un repositorio pequeño y entendible, no una aplicación visual sofisticada.

Semanas 3 y 4: Bedrock directamente, sin LangChain

Ahora reconstruyes el sistema utilizando:

S3 → Bedrock Knowledge Bases → vector store → Bedrock model → API.

Aprende primero la consola, pero después repítelo con Python y boto3 . Debes dominar la diferencia entre:

- RAG completamente administrado con RetrieveAndGenerate .
- Recuperación mediante Retrieve y generación controlada por tu aplicación.
- Knowledge Bases frente a una implementación personalizada.
- Metadata filtering, chunking, embeddings, recuperación y citas.
- On-demand inference, inference profiles y provisioned throughput.

Bedrock Knowledge Bases administra gran parte del flujo de ingestión, recuperación y augmentación, pero AWS también permite evaluar tanto modelos como Knowledge Bases.  AWS Documentat... +2

Semanas 5 y 6: Guardrails y seguridad

Aquí debes detenerte y estudiar este tema como un módulo completo, no como una pestaña.

Tu proyecto debe probar:

- Prompt injection y jailbreaks.
- PII en la pregunta y en la respuesta.
- Temas que el asistente no debe responder.
- Documentos maliciosos introducidos en la base de conocimiento.
- Respuestas sin suficiente evidencia.
- Acceso de diferentes usuarios a documentos distintos.
- Intentos del agente de ejecutar herramientas no autorizadas.

En Bedrock configura content filters, prompt attack detection, denied topics, sensitive-information filters y contextual grounding. Después complementa con IAM de mínimo

privilegio, cifrado, control de fuentes, autorización de herramientas, VPC endpoints y logs. AWS recomienda tratar seguridad, confiabilidad, costos y operación como partes del ciclo completo, no como tareas posteriores al desarrollo.  AWS Documentat... +2

Una respuesta profesional de entrevista sería:

"Implementaría defensa en profundidad. Aplicaría Bedrock Guardrails sobre inputs y outputs para prompt attacks, PII, contenido y temas prohibidos; validaría y aislaría el contexto recuperado; restringiría las herramientas con IAM de mínimo privilegio; registraría invocaciones y construiría un dataset adversarial para medir falsos positivos y falsos negativos."

Eso es muy distinto de decir "vi que Bedrock tiene una pestaña de Guardrails".

Semanas 7 y 8: evaluación, observabilidad y costos

Crea un dataset de al menos 100 preguntas:

- Preguntas respondibles.
- Preguntas sin respuesta en los documentos.
- Preguntas ambiguas.
- Preguntas con información desactualizada.
- Ataques al prompt.
- Preguntas que requieren filtros por metadata.

Mide por separado:

Retrieval: Recall@k, precision@k, MRR y si el fragmento correcto fue recuperado.

Generation: groundedness o faithfulness, relevancia, exactitud, calidad de las citas, rechazo correcto y seguridad.

Operación: latencia p50/p95, tokens, costo por consulta, errores y throttling.

Bedrock ofrece evaluaciones para modelos y Knowledge Bases, y model invocation logging puede registrar inputs, outputs e invocaciones en CloudWatch.

 AWS Documentat... +1

Aquí es donde un proyecto deja de ser "un chatbot que funciona" y empieza a ser un sistema profesional.

Semanas 9 y 10: despliegue real

Despliega el proyecto con:

API Gateway → Lambda → Bedrock → Knowledge Base.

Incluye IAM de mínimo privilegio, CloudWatch, manejo de errores, timeouts, retries, presupuesto y alarmas. Provisiona la infraestructura con AWS CDK o Terraform y agrega CI/CD desde GitHub.

Tu repositorio debe contener:

- Diagrama de arquitectura.
- Código de infraestructura.
- Código de ingestión.
- Código de consulta.
- Dataset de evaluación.
- Resultados antes y después de optimizaciones.
- Threat model.
- Estimación de costos.
- Runbook de incidentes.
- README con decisiones y trade-offs.

AWS mantiene un repositorio oficial de ejemplos que incluye introducción, RAG, embeddings, responsible AI, observabilidad, evaluación y una sección de paso de prueba de concepto a producción. Úsalo como referencia, no como código para copiar sin entender.  GitHub

Semana 11: LangChain y LangSmith

Solo ahora reconstruyes la capa de orquestación con LangChain.

Compara claramente:

- Implementación directa con `boto3`.
- Bedrock Knowledge Bases administrado.
- LangChain usando Bedrock como proveedor.
- LangGraph para un flujo con decisiones o agentes.
- LangSmith para trazas y evaluaciones.

LangChain Academy tiene cursos específicos de observabilidad y evaluación con LangSmith, incluyendo evaluación online, pairwise evaluation y análisis de traces.

 LangChain Acade... +2

En una entrevista debes poder decir:

"No necesito LangChain para implementar RAG en Bedrock. Lo uso cuando su abstracción, integración, trazabilidad o portabilidad compensan la complejidad adicional."

Semana 12: trasladarlo a GCP

No construyas otro proyecto. **Porta el mismo proyecto:**

- S3 → Cloud Storage.
- Bedrock Models → Gemini en Vertex AI.
- Bedrock Knowledge Bases → Vertex AI RAG Engine.
- OpenSearch/vector store → Vertex AI Vector Search o backend compatible.
- Lambda/API Gateway → Cloud Run.
- Bedrock Guardrails → Model Armor y controles de Vertex AI.
- CloudWatch → Cloud Logging y Cloud Monitoring.

Vertex AI RAG Engine es el componente administrado de Google Cloud para incorporar fuentes externas a los modelos, mientras que Model Armor protege prompts, respuestas e interacciones frente a prompt injection, jailbreaks, contenido dañino y filtraciones de datos sensibles.  Google Cloud Do... +1

La ruta oficial **Advanced: Generative AI for Developers** de Google Cloud incluye actividades sobre Vector Search y embeddings, multimodal RAG, privacidad, seguridad y MLOps para IA generativa. Déjala para esta etapa, después de AWS.  Google Skills

El proyecto que deberías construir

Por tu experiencia, construiría un **Fraud and Banking Policy Copilot**.

No usaría el LLM para calcular fraude directamente. El asistente respondería preguntas sobre políticas, procedimientos, tipologías, manuales operativos y protocolos de investigación.

Debe incluir documentos versionados, metadata por fecha/producto/país, citas, control de acceso, RAG administrado y personalizado, Guardrails, dataset de evaluación, análisis de costos, API desplegada, observabilidad y un pequeño agente que consulte una herramienta controlada, por ejemplo una API simulada de casos.

Después implementarías exactamente el mismo caso en GCP. Eso te permitiría explicar Bedrock y Vertex AI desde una sola arquitectura mental.

Cuándo podrás decir que estás listo

Estarás verdaderamente preparado cuando puedas diseñar en una pizarra y defender:

Requisitos → gobierno de datos → ingestión → chunking → embeddings → almacenamiento vectorial → retrieval → reranking → generación → guardrails → evaluación → despliegue → monitoreo → mejora continua.

Y cuando te pregunten “¿has usado Guardrails?”, ya no responderás desde una pestaña. Podrás explicar amenazas, controles, evaluación, limitaciones, costos y decisiones arquitectónicas.

Tu siguiente paso concreto es empezar el curso completo de Bedrock y crear desde el primer día el repositorio del **Banking Policy Copilot**. No le pidas a una IA que genere el proyecto entero. Haz que te acompañe módulo por módulo, exigiéndole siempre explicación, implementación, pruebas, errores, seguridad, evaluación y criterios de aceptación.

     ...  Sources

ok me gusta. pero entiendo que lang chain es un metodo alejado del cloud 6 no estoy seguro de eso. pero si es asi. las empresas estan necesitando rag en AWS con bedrod y opensearch, luego lo miso en GCP. y finalmente lang chain. y no tengo empleo. asi que necesito el cmaino mas afilado y efectivo para lograrlo.

Worked for 42s >

Sí, pero hay una corrección importante que cambia por completo el camino:

LangChain no es otra forma de hacer RAG separada de AWS o GCP. Es una librería de orquestación que puede trabajar encima de Bedrock, OpenSearch, Gemini, Vertex AI, Pinecone y otros servicios. Incluso tiene integraciones específicas para modelos, embeddings y Knowledge Bases de Bedrock.  Docs by LangChain +2

Por eso, el orden más eficiente **no sería**:

AWS → GCP → LangChain

Sería:

RAG esencial → AWS nativo → LangChain sobre AWS → producción → portar a GCP

Las vacantes actuales suelen pedir estas habilidades juntas: Python, RAG, vector databases, Bedrock, OpenSearch y algún framework como LangChain, LangGraph o Strands. No esperan necesariamente tres especializaciones aisladas.  Lever +3

Tu objetivo profesional inmediato

No necesitas convertirte ahora en experto absoluto en toda la IA generativa.

Tu perfil objetivo debería ser:

AWS Generative AI / RAG Engineer con Python, Bedrock, OpenSearch, LangChain y experiencia desplegando soluciones empresariales.

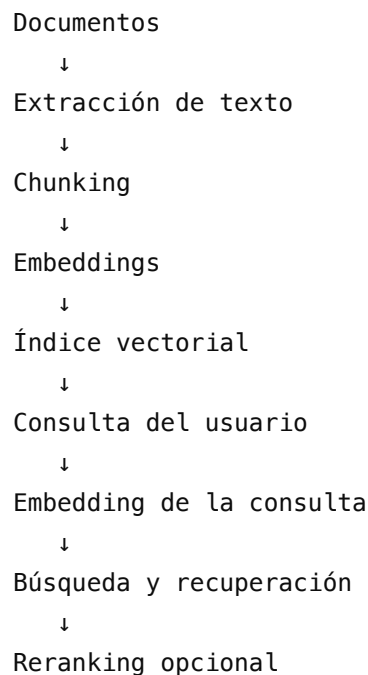
GCP será tu segunda nube, pero no debe retrasar tu capacidad de postularte.

Tu primera meta no es “aprender todo”. Es poder demostrar que sabes construir, explicar, asegurar, evaluar y desplegar **un sistema RAG profesional en AWS**.

La ruta más afilada

Etapa 1: fundamentos de RAG — una semana

Antes de profundizar en Bedrock, necesitas controlar mentalmente este flujo:



```
graph TD; A[Documentos] --> B[Extracción de texto]; B --> C[Chunking]; C --> D[Embeddings]; D --> E[Índice vectorial]; E --> F[Consulta del usuario]; F --> G[Embedding de la consulta]; G --> H[Búsqueda y recuperación]; H --> I[Reranking opcional];
```

Documentos

↓

Extracción de texto

↓

Chunking

↓

Embeddings

↓

Índice vectorial

↓

Consulta del usuario

↓

Embedding de la consulta

↓

Búsqueda y recuperación

↓

Reranking opcional



↓
Prompt con contexto
↓
LLM
↓
Respuesta con fuentes

En esta semana no debes construir infraestructura compleja. Puedes usar Python, archivos PDF o Markdown, una base vectorial local y una API de modelo.

Lo importante es dominar:

- Chunk size y overlap.
- Embeddings.
- Similitud vectorial.
- Top-k.
- Búsqueda semántica, keyword e híbrida.
- Metadata filtering.
- Reranking.
- Context window.
- Respuestas con citas.
- Alucinaciones y ausencia de información.
- Evaluación separada del retrieval y la generación.

No debes invertir más de una semana aquí. Ya eres ingeniero de IA; no necesitas un curso introductorio de meses.

Etapas 2: AWS Bedrock nativo — dos semanas

Este debe ser tu núcleo principal.

Construirás el mismo sistema utilizando:

S3
↓
Bedrock Knowledge Bases
↓
Titan Embeddings
↓
OpenSearch Serverless
↓
Retrieve o RetrieveAndGenerate
↓
Claude u otro modelo en Bedrock



↓
FastAPI

Bedrock Knowledge Bases puede crear y configurar automáticamente una colección e índice de OpenSearch Serverless. También puedes construir y controlar tú mismo el vector store.  AWS Documentat... +1

Debes aprender dos implementaciones.

Implementación administrada:

```
bedrock_agent_runtime.retrieve_and_generate(...)
```

Aquí Bedrock se ocupa del retrieval, construcción del prompt y generación. La API también administra sesiones conversacionales.  AWS Documentat... +1

Implementación controlada:

```
results = bedrock_agent_runtime.retrieve(...)
prompt = build_prompt(question, results)
response = bedrock_runtime.converse(...)
```

Aquí controlas el prompt, los fragmentos recuperados, la generación, las fuentes y la lógica de rechazo. Retrieve también permite aplicar filtros sobre metadata.

 AWS Documentat...

Para entrevistas, debes saber explicar cuándo usarías cada una:

- RetrieveAndGenerate : implementación rápida y administrada.
- Retrieve + Converse : mayor control, testing, guardrails, personalización y observabilidad.
- OpenSearch directamente: control completo sobre mappings, hybrid search, filtros, índices y consultas.

Al terminar estas dos semanas ya deberías empezar a aplicar a vacantes.

Etapas 3: LangChain sobre el mismo proyecto — una semana

No construyas un proyecto diferente.

Toma tu aplicación AWS y reemplaza algunas partes con:

- ChatBedrockConverse .
- BedrockEmbeddings .
- AmazonKnowledgeBasesRetriever .
- Chains para el flujo RAG.
- LangSmith o equivalente para tracing.

LangChain tiene componentes oficiales para utilizar modelos, embeddings y retrievers de Bedrock. Incluso LangSmith puede trazar llamadas realizadas directamente con el SDK nativo de AWS, por lo que puedes usar observabilidad sin reescribir toda tu aplicación con LangChain.  Docs by LangChain +2

Tu repositorio debería mostrar dos ramas o dos implementaciones:

```
/native
  boto3 + Bedrock + OpenSearch

/langchain
  LangChain + Bedrock + OpenSearch
```



Eso es extremadamente valioso para entrevistas porque puedes decir:

“Primero implementé el flujo directamente con Boto3 para comprender y controlar cada componente. Después construí una segunda implementación con LangChain para comparar velocidad de desarrollo, observabilidad, portabilidad y complejidad.”

Eso demuestra mucho más conocimiento que hacer únicamente un tutorial de LangChain.

Etapas 4: seguridad, evaluación y producción — dos semanas

Esta etapa es la que más te diferenciará.

Tu proyecto debe incorporar Bedrock Guardrails para:

- Prompt attacks.
- Contenido dañino.
- Temas prohibidos.
- Información sensible.
- Grounding.
- Respuestas fuera de contexto.

Pero también debes agregar seguridad fuera de Guardrails:

IAM de mínimo privilegio
Cifrado
Separación por usuario o tenant
Metadata filtering
Validación de documentos
Control de acceso a fuentes
Logging
Rate limiting
Protección de herramientas



También necesitas un dataset de evaluación, por ejemplo 80 o 100 preguntas divididas en:

- Respondibles.
- No respondibles.
- Ambiguas.
- Fuera del dominio.
- Con filtros de metadata.
- Adversariales.
- Prompt injection.
- Información sensible.

Mide:

Retrieval:
Recall@k
Precision@k
MRR
Hit rate

Generación:
Faithfulness
Answer relevance
Correctness
Citation accuracy
Refusal accuracy

Operación:
Latencia
Costo
Tokens
Errores
Throttling



Las vacantes actuales no se limitan a “crear un chatbot”; mencionan observabilidad, evaluación, Human-in-the-Loop, guardrails, infraestructura serverless y sistemas listos para producción.  Lever +2

Etapla 5: GCP — dos semanas

Solo después de tener el proyecto AWS terminado.

No vuelvas a estudiar RAG desde cero. Porta la misma solución:

AWS	GCP
Amazon S3	Cloud Storage
Amazon Bedrock	Vertex AI / Gemini
Bedrock Knowledge Bases	Vertex AI RAG Engine
OpenSearch Serverless	Vertex AI Vector Search
Lambda	Cloud Run o Cloud Functions
API Gateway	API Gateway
Bedrock Guardrails	Model Armor y controles de aplicación
CloudWatch	Cloud Logging y Monitoring
IAM	Cloud IAM

Actualmente Google ofrece RAG Engine integrado con su infraestructura de búsqueda vectorial, incluyendo opciones administradas para embeddings, corpus y recuperación.

 Google Cloud Do... +3

No necesitas inicialmente dominar todo Vertex AI. Para empleabilidad enfócate en:

Gemini API
Vertex AI SDK
Cloud Storage
RAG Engine
Vector Search
Cloud Run
IAM
Secret Manager



Logging
Model Armor

El único proyecto principal que necesitas

Construye un **Enterprise Banking Knowledge Assistant**.

Usa documentos públicos o ficticios sobre:

- Políticas bancarias.
- Prevención de fraude.
- Procedimientos operativos.
- Reclamaciones.
- Transferencias.
- Seguridad digital.
- Manuales de productos.

Debe tener:

1. Ingestión de documentos desde S3
2. Parsing y chunking
3. Titan Embeddings
4. OpenSearch Serverless
5. Bedrock Knowledge Base
6. RetrieveAndGenerate
7. Retrieve + Converse personalizado
8. Respuestas con citas
9. Filtros por metadata
10. Bedrock Guardrails
11. Evaluación automática
12. FastAPI
13. Autenticación básica
14. CloudWatch
15. Infraestructura como código
16. Docker
17. CI/CD
18. Estimación de costos
19. Diagrama de arquitectura
20. Versión equivalente con LangChain



Después lo portas a GCP.

No construyas cinco chatbots pequeños. **Construye uno solo y llévalo progresivamente a nivel profesional.**

Cronograma intensivo de ocho semanas

Suponiendo unas cuatro horas diarias:

Semana	Resultado
1	RAG desde cero sin framework
2	Bedrock, S3 y Knowledge Bases
3	OpenSearch y RAG controlado con Boto3
4	LangChain integrado con Bedrock
5	Guardrails, seguridad y ataques
6	Evaluación, observabilidad y despliegue
7	Portabilidad a GCP
8	Entrevistas, documentación y aplicaciones

No esperes a la semana ocho para buscar empleo.

Desde la semana tres puedes colocar en tu perfil:

Building a production-oriented RAG system with Amazon Bedrock, Knowledge Bases, OpenSearch Serverless, Titan Embeddings, Boto3 and FastAPI.

Desde la semana cuatro:

Implemented and compared native Bedrock and LangChain-based RAG architectures.

Desde la semana seis:

Added guardrails, evaluation datasets, observability, metadata-based access control and serverless deployment.

Qué estudiar y qué ignorar por ahora

Estudia inmediatamente:



- Python
- FastAPI
- Boto3
- Bedrock Runtime
- Bedrock Knowledge Bases
- Titan Embeddings
- OpenSearch
- S3
- IAM
- Lambda
- API Gateway
- CloudWatch
- Guardrails
- LangChain
- RAG evaluation
- Docker
- Terraform o CDK

Déjalo para después:



- Fine-tuning avanzado
- Entrenamiento de LLM desde cero
- Kubernetes avanzado
- Multi-agent systems complejos
- CrewAI
- Decenas de vector databases
- Tres o cuatro frameworks simultáneamente
- RAG multimodal avanzado
- Graph RAG profundo

Cómo estudiar sin volver a perderte

No le pidas a una IA:

“Créame un proyecto completo de RAG profesional.”

Eso produce demasiado código y poca comprensión.

Trabaja mediante entregables pequeños:



- Módulo 1: ingestión
- Módulo 2: chunking
- Módulo 3: embeddings
- Módulo 4: OpenSearch
- Módulo 5: retrieval
- Módulo 6: generación

Módulo 7: citas
Módulo 8: guardrails
Módulo 9: evaluación
Módulo 10: despliegue

En cada módulo debes exigir:

Explicación arquitectónica
Código
Pruebas
Errores frecuentes
Seguridad
Costo
Preguntas de entrevista
Criterios de aceptación



Mi recomendación definitiva

Tu prioridad debe quedar así:

1. RAG esencial
2. AWS Bedrock nativo
3. OpenSearch
4. LangChain integrado con AWS
5. Guardrails y evaluación
6. Despliegue profesional
7. GCP equivalente
8. LangGraph y agentes



No pongas LangChain al final de todo, porque actualmente aparece combinado con Bedrock, RAG y vector databases en las mismas posiciones. Pero tampoco empieces por LangChain, porque corres el riesgo de aprender nombres de clases sin comprender la arquitectura subyacente.  Lever +3

La estrategia correcta para conseguir empleo es:

profundidad inmediata en AWS, conocimiento práctico de LangChain y capacidad posterior de trasladar la arquitectura a GCP.

¿Te configuro un monitoreo diario de vacantes remotas que mencionen RAG, Bedrock, OpenSearch, LangChain o Vertex AI?

       Sources

Do you like this personality?   